

Web-Technologies Project 2025

1. Overview

Your team will develop a **food-delivery platform** (a simplified version of popular online ordering services). The system connects three main actors:

Actor	Responsibility
Site Manager	Oversees the whole platform, manages global settings, monitors activity.
Restaurant Owner	Lists menus, receives orders, and manages order fulfillment.
Customer (User)	Browses restaurants, places orders, tracks delivery, and gives feedback.

All development is to be completed using **Angular** for the frontend, **Node.js** (REST API) for the backend, and an **SQL database** for data persistence. **PostgreSQL is recommended**, but other SQL databases (MySQL, MariaDB, SQLite, etc.) are allowed.

2. Grouping

If the total number of students is **not divisible by three**, the following fallback rules apply:

- **Groups with 4 students** – One module can be implemented collaboratively by two students:
 - Both students must contribute meaningfully to the code and documentation
 - Each student must take on at least **one more extra task**, in addition to the one that is recommended for everybody to reach the full 40% extra task score
- **Groups with 2 students** – The students implement only the following roles, but still deliver a fully functional system:
 - Restaurant Owner
 - Customer (User)
- **Single leftover students** are strongly encouraged to **join an existing group**. In **exceptional cases**, a student may complete the project individually. Requirements for individual work:
 - Instructor approval is required
 - Implement all common functionalities
 - Implement one full role module
 - **No extra tasks** are required to achieve the maximum (100%) grade

3. Common Functionalities

All participants share responsibility for implementing the core infrastructure that every actor uses.

Feature	Description
User Registration & Authentication	Email + password sign-up, password reset, address information. No distinction between roles at login – different user credentials grant access to the appropriate UI based on the user's role.
Profile Management	Users can edit personal data, change passwords, and (restaurant owners) update restaurant contact details.
Responsive UI	Layout adapts to smartphones, tablets, and desktops.
Security Basics	Prepared statements, input validation and sanitization, no exposure of sensitive data, other lecture-taught concepts.
Database Layer	SQL schema design and seed data (sample restaurants, dishes, and users).
REST API	CRUD endpoints for all entities, JWT-based authentication, proper HTTP status codes.
Shared UI Components	Navigation bar, user notifications, loading screens, user information, error handling dialogs.

These items must be documented in a PDF report.

4. Basic Tasks – Individual core modules per student

Each group ideally consists of **three students**, with each student responsible for one of the following core modules.

Note: Every student must implement the basic tasks for at least one module to meet learning outcomes. Clear documentation of who implemented which parts is mandatory.

Note: To support essential features such as delivery time estimation, restaurant filtering, and delivery-zone validation, every project must implement the basic location and distance simulation described in the Addendum (Section 10.1). Real geolocation services are optional (extra task), but a consistent simulation method must be used for mandatory functionality.

4.1. Site Manager

- **Dashboard**
 - Overview of all active restaurants
 - Pending restaurant registration requests
 - Key statistics: total orders, revenue, user activity
 - **Global Settings**
 - Configure allowed restaurant food delivery zones/areas
 - Set service fees for restaurants using the platform
 - Set up platform-wide vouchers/promotional codes (e.g. 5% discount with code "FOOD2026")
 - **User & Restaurant Moderation**
 - Approve or reject new restaurant accounts
 - Suspend or warn abusive users
 - **Reporting**
 - Generate platform-wide reports for orders and revenue
 - Track user activity trends over time (e.g. logins, changes, etc.)
-

4.2. Restaurant Owner

- **Menu Management**
 - Create, edit, and delete categories and dishes
 - Manage dish details: name, description, price, optional photo
 - **Order Reception**
 - View incoming orders in real-time
 - Accept or reject orders
 - Update order status: preparing → ready → dispatched
 - **Restaurant Profile**
 - Set restaurant name and contact info
 - Define opening hours
 - Pick delivery zone/area (predefined by site manager)
 - **Analytics**
 - Display basic statistics: daily/weekly order counts
 - Identify most-ordered dishes
-

4.3. Customer (User)

- **Restaurant Browsing**
 - Filter restaurants by cuisine, rating, delivery time, or distance (if implemented)
 - View restaurant details and full menu
- **Cart & Checkout**
 - Add, modify, or remove dishes from the cart
 - Apply vouchers/promotional codes
 - Place an order
- **Order Tracking**
 - View current status of order
 - Display estimated arrival time
(See [Addendum 10.1](#) for required location and distance simulation methods)
- **Feedback**
 - Rate restaurants and individual dishes
 - Leave textual reviews

Note: Each role must include a **dedicated Angular module** and corresponding backend routes. These basic tasks comprise **60% of the grade**.

5. Extra Tasks – Distributed Across All Groups

Beyond the mandatory modules, it is recommended that each group member takes on one **extra task**. Extra tasks can contribute up to **40% of the grade**, but a positive grade can still be achieved without them (see [9. Evaluation Criteria](#)). Refer to [Addendum: 10.2](#) for the full list of available extra tasks.

Note: All extra tasks must be documented, with clear assignment of responsibility among team members.

6. Deliverables

6.1. Source Code Repository

The source code should:

- Be hosted on GitHub/GitLab/Bitbucket
 - Have a clear commit history (showing individual contributions)
 - Include a separate Angular frontend + Node.js backend + SQL schema
-

6.2. Documentation PDF

The documentation must include:

- Project overview
 - a. Short description of the platform
 - b. System architecture diagram (frontend–backend–database)
 - Database schema
 - a. Class diagram
 - b. Table descriptions & relationships
 - c. Seed data description
 - Module Responsibilities – description of what each student implemented
 - a. Site Manager, Restaurant Owner, Customer (User) modules
 - b. Any supporting shared components or backend services
 - Extra Tasks
 - a. Which extra tasks were implemented
 - b. Who implemented them
 - c. Short reflection on complexity & challenges
 - Setup Instructions
 - a. How to install dependencies
 - b. How to run backend + frontend
 - c. How to import database schema
 - d. Any environment variables (no secrets!) should be listed
-

7. Presentations

The presentations are split into the following parts.

7.1. Vision Presentation (15 | 18 December)

Goal: Show the project concept and feasibility

Each group presents:

- Vision & feature overview
- Mockups or UI sketches (wireframes)
- Planned **database schema**
- Planned **architecture** (Angular modules, backend routes, tables, services)
- Distribution of responsibilities within the team
- Planned extra tasks (optional)

Duration: ~10 min

7.2. Mid-Term Presentation (12 | 15 January)

Goal: Proof of active work and understanding

Students must show:

- Progress after 3 weeks of work (including Christmas break)
- Early working demo or technical prototype
- At least:
 - Working login + initial module structure
 - First endpoints
 - Database schema implemented
- Clear signs of real teamwork (commits, code structure, screenshots)

Focus: We want to verify that you have started coding and understand the framework basics.

7.3. Final Presentation (26 | 29 January)

Goal: Demonstrate full functionality and frontend

Students must show:

- All basic features working for their assigned role
- Clean navigation, UI flow, and responsive behavior
- Basic integration between Angular frontend and Node backend
- Minimal demo of real data flow (create order → update → view)

This presentation is feature-focused (what works in the final solution).

Duration: ~10–15 min.

7.4. Final Submission Talk – Abgabegespräch (February)

This is the most important part.

Focus:

- **Understanding of Angular & Node frameworks**
- Not the feature set, but the *knowledge behind the code*

Every student must:

- Explain *their own code* clearly
- Be able to answer questions such as
 - “Explain how to create a new Angular module”
 - “Where would you add a new route in the backend?”
 - “What is the purpose of app.module.ts / routing modules / guards?”
 - “Why does Angular generate a component with these files?”
 - “Explain how dependency injection works here”
 - “What does this middleware in Express do?”
- Provide reasoning behind architectural choices

Students do **not** have to write code live, but they must demonstrate understanding. If a student cannot explain the code, the module cannot be graded positively.

8. Schedule

Dates Hollauf / Leibetseder	Dates Shekhovtsov	Event	Notes
24 Nov 2025	27 Nov 2025	Project release + group formation call	In session before 2nd exam
01–11 Dec 2025		Final group confirmation	Students who cannot pass via project will be removed
12 Dec 2025		Tutorium	Voluntary Project Tutorium 17:00-18:30, online - see Lecture Moodle Course BBB
15 Dec 2025	18 Dec 2025	Vision Presentation	Mockups, architecture, DB schema, team roles
12 Jan 2026	15 Jan 2026	Mid-Term Presentation	Proof of work over Christmas, technical progress
26 Jan 2026	29 Jan 2026	Final Presentation	Show full functionality
01 Feb 2026		Project Deadline	Full code + documentation
09–13 Feb 2026		Submission Interviews (Week 2)	Framework-competence oral examination
15 Feb 2026		Optional grace period deadline	For late or incomplete submissions
23–27 Feb 2026		Late Submission Interviews (Week 4)	For grace period submissions

9. Evaluation Criteria

Total Points: 40

- **Basic Tasks (60% - 25P)** – Completion and correctness of all mandatory features.
- **Extra Tasks (40% - 15P)** – Extra features, optional tests, deployment, and advanced improvements.
- **Positive grade (50% - 20P) achievable** with basic tasks alone.

Note: All team members should attempt at least one extra task, but a positive grade can be achieved without completing any extra task.

10. Addendum

10.1. Handling Locations and Distances

To support essential features such as estimated delivery time, restaurant filtering, and delivery-zone validation, every project must implement a **basic location and distance simulation**.

Real geolocation services (Google Maps, OpenStreetMap, etc.) are **not required** (unless chosen as an **extra task**), but each team must choose **one method** for representing locations and for calculating distances or travel durations.

Restaurant owners will assign a delivery time when accepting new orders. This time must be **computed or influenced** by the chosen simulation method (e.g., distance categories, travel-time rules, or predefined zones), not simply set as an arbitrary static value.

Projects are free to design their own model, but the following approaches are suggested and fully acceptable:

10.1.1. Option A – Grid-Based Coordinates

- Assign fictional coordinates to users and restaurants (e.g., (x, y))
- Compute distances such as Manhattan distance:
 $Distance = |x_1 - x_2| + |y_1 - y_2|$
- Map distance to delivery time categories
e.g., $\leq 3 \rightarrow 20 \text{ min}$, $\leq 6 \rightarrow 35 \text{ min}$, $> 6 \rightarrow 50 \text{ min}$

10.1.2. Option B – Area/Zone Labels

- Use symbolic areas like A1, B2, etc.,
- Derive proximity via row/column differences:
 - Exact same code $\rightarrow$ very close
 - Same row or column $\rightarrow$ nearby
 - Different row and column $\rightarrow$ far

10.1.3. Option C – Named Delivery Zones

- Predefine zones such as *North*, *South*, *Central*, *Industrial*.
- Each zone has associated delivery rules or typical durations.

10.1.4. Option D – Radius-Based Simulation

- Assign coordinates and compute Euclidean distance.
- Accept or reject orders based on a configurable maximum radius.

Any consistent and well-documented method is accepted.

At minimum, your implementation must:

1. Assign a location representation to users and restaurants.
2. Compute a simple form of “distance” or “travel effort.”
3. Derive an estimated delivery time from this simulation.
4. Use this information in relevant UI components (e.g., filtering by proximity or time, sorting restaurants, order estimation).

Note on Extra Task “Geolocation & Maps”:

If any team member chooses the extra task that introduces an actual map or uses real coordinates, the above logic must be adapted accordingly. A real map grid or a simple simulated grid both remain valid approaches, i.e. map integration can still be done with simulated, non-real coordinates unless the team voluntarily chooses to use real-world map data.

10.2. Extra Tasks

Category	Task
Real-time Communication	Implement a WebSocket-based chat between customers and restaurant staff for order questions, and show live order status updates (e.g., order accepted → preparing → ready → delivered) in real time. Clarification: The chat should support only one-to-one communication, implementing multiple participants is not required.
Notifications & Email Integration	Implement push notifications to customers for order status changes (accepted, ready, delivered) and send transactional emails with order confirmation and receipts (including order details as attachments) to both customers and restaurants. This should also include support for sending promotional emails to the customers with a possibility to opt-out (allowing the users to specify that they do not want to receive any further promotional emails). The promotional emails should contain the text messages entered by the administrator using a UI form, they should be sent to all customers who did not refuse to receive them.

Geolocation & Maps	Display restaurant locations and delivery zones on a map or custom grid. Implementation may use OpenStreetMap, Google Maps, or a custom coordinate/grid system using any framework e.g. Leaflet or Angular/Google-maps. Distances or travel times must be computed consistently for filtering and delivery time estimation as described in Addendum 10.1.
Offline Support / Cart Persistence	Cache menu data and restaurant information locally to allow offline browsing and cart building. Automatically synchronize the cart when back online. This can be implemented using localStorage, IndexedDB, cookies, or a service worker with the Cache API, combined with server-side synchronization for more robust persistence. Clarification: Only a limited offline snapshot is required. Students may either (A) cache restaurants/menus the user has already viewed, or (B) pre-cache a small subset (e.g., last visited restaurant). When offline, the user can browse only this cached data and continue building the cart. When back online, the cart is synchronized normally and the complete data has to be available again. Full platform-wide offline browsing is not required. The whole app is supposed to be launched online, as usual, it should only be able to handle the case when the internet is off when it is running. To detect if the internet is on or off, the app may query its backend repeatedly in a background task (a watchdog task).
Drag-and-Drop & File Upload	Implement a system for restaurant owners to upload images and manage menus interactively. This includes uploading banners or logos with live previews, handling multiple dish photos with previews and removal options, and reordering menu categories via drag-and-drop, with changes persisting in the database and reflected in the UI. Live previews can use real photos or placeholders. Clarification: The only menu-related enhancement required by this extra task is drag-and-drop reordering of categories (and optionally dishes). All other listed features relate to image upload and interactive UI behavior.
Forum / Community Board	Implement a discussion board where users can post questions about restaurants, and reply to the questions posted by others, the discussion should be organized by threads, each thread starting from a specific posted question. The board should include moderation functionality for restaurant owners.

Large Data Handling & Pagination	Implement efficient display and navigation for large restaurant and dish datasets. Include server-side sorting, filtering by a single or multiple fields, and either server-side pagination or infinite scrolling/“load more” functionality. Optionally, caching or prefetching may be used to improve performance. Angular material components such as data tables and a paginator component may be used for implementing tabular display with sorting and pagination. Clarification: Server-side pagination means that the information about the current page number has to be sent to the backend as a part of the HTTP request, and a list of results corresponding to this page is a part of the response. By default, Angular material data tables with a paginator component support client-side pagination when the whole result set is given to the table component, which shows it a page at a time. To make them support server-side pagination, it is necessary to send the new request to the server on every page change, and provide the result to the table component.
Loyalty & Rewards	Implement a loyalty system where customers earn points for each order, view their point balance, redeem points for rewards (discounts or free items), and track reward history in their profile. Include visual indicators in the UI for available rewards and active promotions. Clarification: • Earning points should be based on the total sum of the goods in the order (e.g. a point for each 10 EUR of the order amount). • “Active promotions” refers to temporary multipliers applied while earning points (e.g., orders on weekends earn 2x points) or limited-time reward offers that can be redeemed with points, all such promotions depend on the calendar date. • “Rewards” to be redeemed with points may include: fixed-amount discounts (e.g., 5€ off for a specific product), percentage discounts (e.g., 10% off for a specific product), or free products chosen by the restaurant owner, each reward should cost a specific number of points (i.e. the free product reward amounts for purchasing the product only for points). The system does not require advanced financial logic – a simple points-per-order model and a small set of predefined reward types is sufficient.

Personalized Recommendations	Implement a recommendation system that suggests restaurants or dishes to customers based on their previous orders, previous restaurant choices, or user ratings. Recommendations should appear on the homepage and restaurant pages, updating dynamically as the user interacts with the platform (i.e. as far as the user makes choices, the rules should take these choices into account and change the list of recommended items accordingly). Clarification: This task does not require knowledge of machine learning or advanced mathematics. A simple set of rules to provide recommendations is sufficient. Students may base recommendations on: • previously ordered dishes, • previous restaurant choices, or • user ratings. Such a set may contain e.g. the following rules: “recommend the most frequently ordered dish by this customer” (or N most frequently ordered dishes), “recommend a random subset of dishes which were in the last N orders of this customer”, “recommend the top-rated restaurants previously visited by this customer”, or “recommend the top-rated restaurants from the city/province of this customer”. An approach based on applying the set of rules defined as above is enough; no model training or complex algorithms are expected.
-------------------------------------	---